

# SECURE CI/CD PIPELINE

Azure DevOps + GitHub + Automated Security Gates

Cybersecurity Portfolio Project

SAST

SCA

Secrets

Docker

Container Scan

**Ghaith Ben Hafsia**

Application Security / Vulnerability Management / DevSecOps

## PROJECT FLOW

Git push



Azure Pipeline



Security scans



Docker build



Security gate

# Project Overview

A hands-on Secure SDLC lab that integrates security scanning directly into an Azure DevOps CI/CD workflow.

## Goal

- Trigger security checks automatically on every Git push or pull request.
- Detect insecure code, vulnerable dependencies, exposed secrets, and vulnerable container packages.
- Block the pipeline when High or Critical security findings are detected.
- Use findings to practice vulnerability triage, remediation, rebuild, and rescan.

## Repository

[github.com/ghaithben07/acr-build-helloworld-node](https://github.com/ghaithben07/acr-build-helloworld-node)

## Technology Stack

|                       |                                             |
|-----------------------|---------------------------------------------|
| ● <b>Azure DevOps</b> | Pipeline orchestration & CI/CD              |
| ● <b>GitHub</b>       | Source control & trigger source             |
| ● <b>Semgrep</b>      | SAST / code & Dockerfile rules              |
| ● <b>Trivy</b>        | SCA, IaC & container vulnerability scanning |
| ● <b>TruffleHog</b>   | Secret scanning                             |
| ● <b>Docker</b>       | Container build and image packaging         |
| ● <b>Node.js</b>      | Sample web application                      |

# Pipeline Architecture

Security is embedded into the build workflow as automated gates, rather than being added only at the end.



## Detailed execution flow

- |   |                   |                                                    |   |                     |                                                       |
|---|-------------------|----------------------------------------------------|---|---------------------|-------------------------------------------------------|
| 1 | <b>Checkout</b>   | Pull the repository and full Git history           | 5 | <b>TruffleHog</b>   | Detect verified or unknown secrets                    |
| 2 | <b>Node setup</b> | Install Node.js and application dependencies       | 6 | <b>Docker build</b> | Create the application container image                |
| 3 | <b>Semgrep</b>    | Static analysis; fail on blocking findings         | 7 | <b>Trivy image</b>  | Fail on High/Critical container vulnerabilities       |
| 4 | <b>Trivy fs</b>   | Scan dependencies and infrastructure configuration | 8 | <b>Deploy gate</b>  | Deployment allowed only after security stage succeeds |

# Azure Pipeline Configuration

The YAML file defines triggers, security tools, failure thresholds, and the deployment gate.

```
trigger:
  branches:
    include:
      - main
      - master

pool:
  vmImage: ubuntu-latest

- script: |
  semgrep scan --config auto --error .
  displayName: SAST - Semgrep

- script: |
  trivy fs --scanners vuln,config \
  --severity HIGH,CRITICAL --exit-code 1 .
  displayName: SCA and IaC Scan - Trivy

- script: |
  docker build -t $(imageName)$(Build.BuildId) .
  displayName: Build Docker Image

- script: |
  trivy image --severity HIGH,CRITICAL \
  --exit-code 1 $(imageName)$(Build.BuildId)
  displayName: Container Scan - Trivy
```

## Key design choices

**Branch triggers** Runs automatically for main/master changes.

**Fail-fast SAST** Semgrep uses --error to block on matching findings.

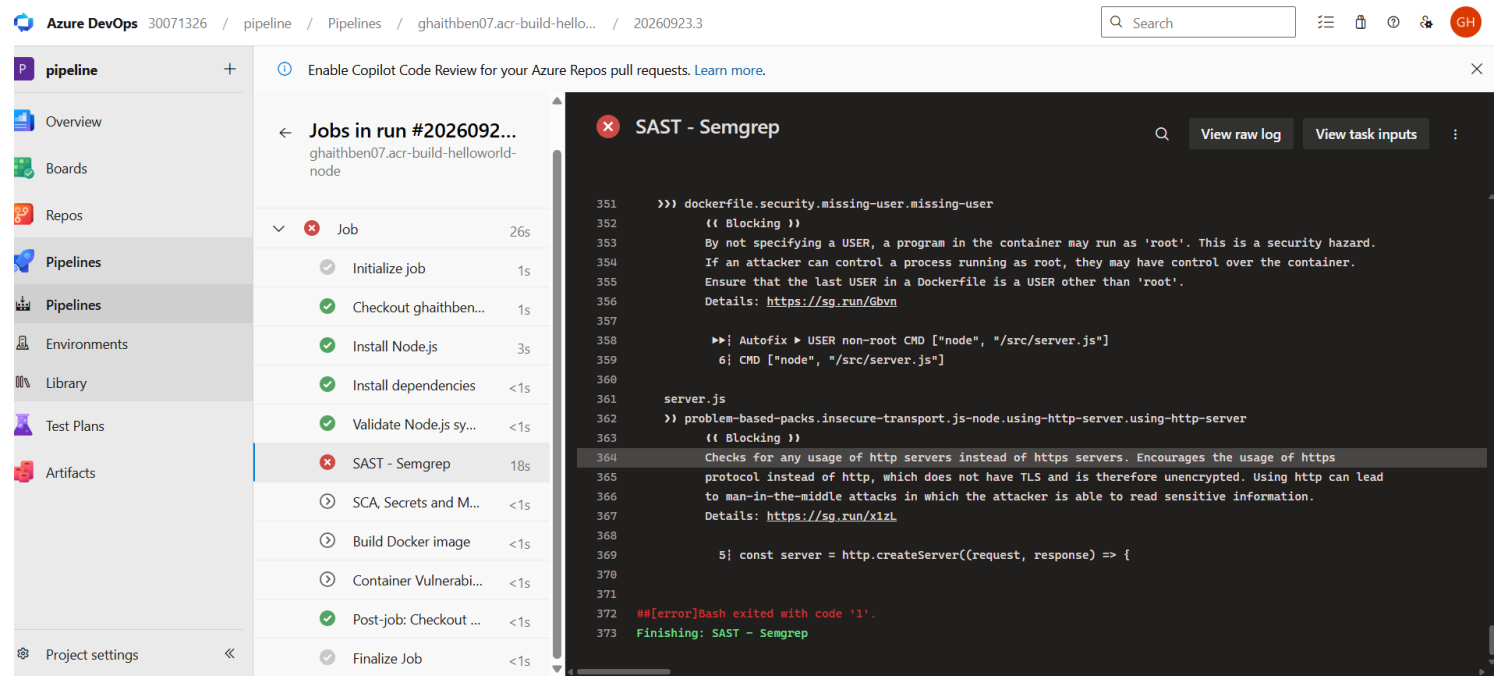
**Risk threshold** Trivy exits with code 1 for High/Critical findings.

**Layered coverage** Code, dependencies, secrets, IaC, and images are scanned.

**Deployment gate** Deploy stage depends on successful security checks.

# Finding 1 - SAST / Semgrep

The pipeline correctly stopped when Semgrep identified blocking security issues in the Dockerfile and Node.js server.



Evidence from Azure DevOps run

## What Semgrep found

Blocking finding

### 1. Dockerfile missing USER

The container could run as root, increasing impact if the process is compromised.

### 2. HTTP server without TLS

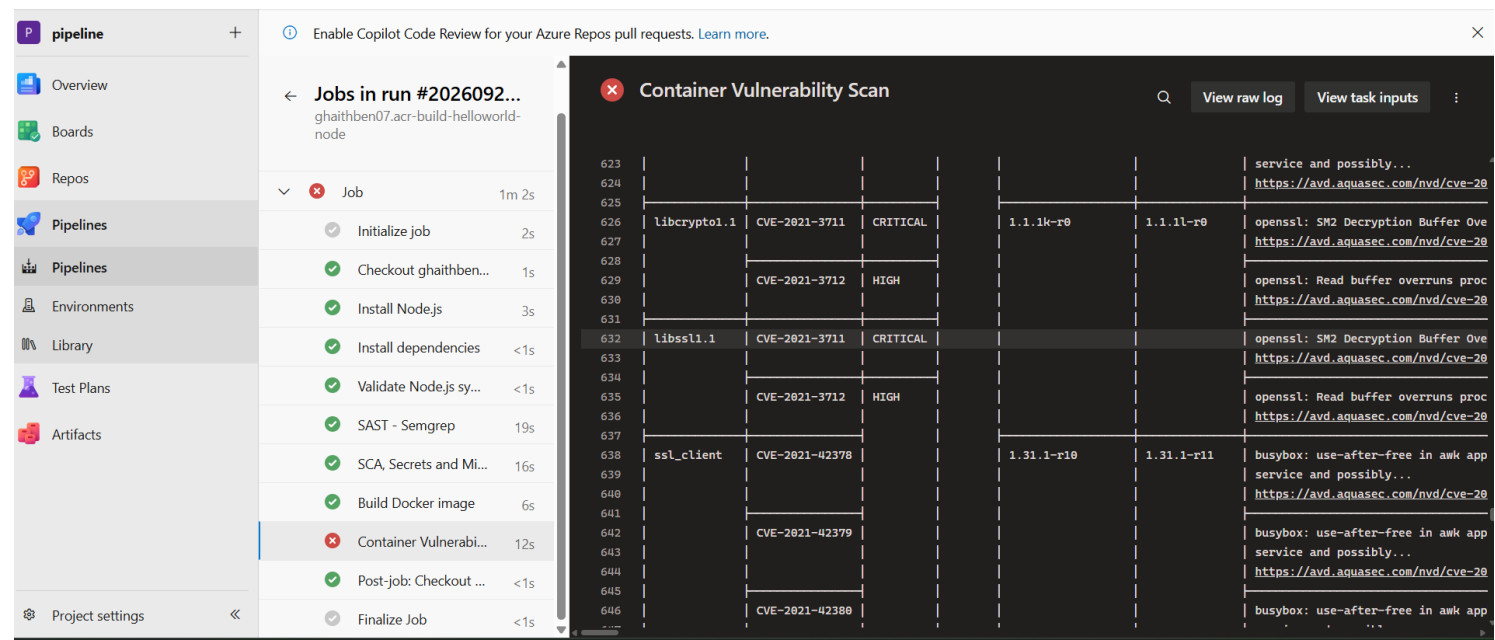
Semgrep flagged plaintext HTTP usage. This requires architecture context and secure deployment design.

### Triage / remediation

Add a non-root USER, review the HTTP finding in deployment context, then rerun the pipeline.

# Finding 2 - Container Vulnerability Scan

After the earlier stages passed, Trivy blocked the build because the container image contained High and Critical CVEs.



Evidence from Azure DevOps run

## Observed examples

**CRITICAL** CVE-2021-3711

OpenSSL / libcrypto & libssl packages shown in the scan output.

**HIGH** CVE-2021-3712

The scan also reported other vulnerable BusyBox / SSL-related packages.

## Remediation approach

- Update the outdated container base image / packages.
- Rebuild the image.
- Rescan with Trivy.
- Do not deploy until the risk is reduced or formally accepted.

# Vulnerability Triage & Remediation Workflow

The lab demonstrates that scanner output is the start of the process - findings must be validated, prioritized, fixed, and retested.



**Security gate behavior**

**High/Critical finding -> non-zero exit code -> Azure job fails -> later deployment stage does not run.**

This turns the CI/CD pipeline into an enforcement control, not only a reporting mechanism.

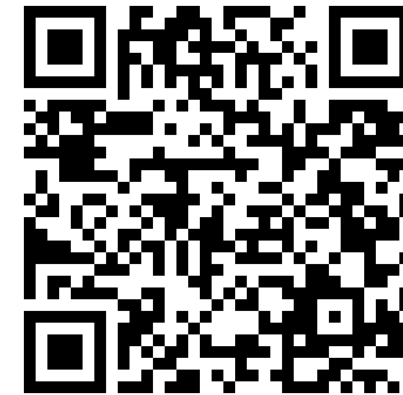
# Project Outcomes

What this project demonstrates to an AppSec, Vulnerability Management, or DevSecOps hiring team.

## Skills demonstrated

- Built and configured an Azure DevOps YAML CI/CD pipeline.
- Integrated SAST, SCA/IaC, secrets scanning, and container vulnerability scanning.
- Created security gates that stop builds on High/Critical findings.
- Triaged SAST findings and container CVEs using tool output and deployment context.
- Practiced remediation -> rebuild -> rescan as part of Secure SDLC.
- Used Git/GitHub, Docker, Node.js, Azure DevOps, Semgrep, Trivy, and TruffleHog.

## GitHub Repository



Scan to open the project repository

**[ghaithben07/acr-build-helloworld-node](#)**

## Portfolio-ready description

Secure CI/CD Pipeline - Azure DevOps

# Appendix - azure-pipelines.yml (1/2)

The exact pipeline configuration used for the project.

```
1 trigger:
2   branches:
3     include:
4       - main
5       - master
6
7 pr:
8   branches:
9     include:
10      - main
11      - master
12
13 pool:
14   vmImage: ubuntu-latest
15
16 variables:
17   imageName: secure-node-app
18
19 stages:
20   - stage: BuildAndSecurity
21     displayName: Build and Security Checks
22     jobs:
23       - job: SecureBuild
24         displayName: Secure Build
25         steps:
26           - checkout: self
27             fetchDepth: 0
28
29           - task: NodeTool@0
30             displayName: Use Node.js 20
31             inputs:
32               versionSpec: 20.x
33
34           - script: npm install
35             displayName: Install Node.js dependencies
36
37           - script: |
38               python3 -m pip install --upgrade pip
39               python3 -m pip install semgrep
40               semgrep scan --config auto --error.
41             displayName: SAST - Semgrep
42
43           - script: |
44               curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
```

# Appendix - azure-pipelines.yml (2/2)

Security scanning, Docker build, container scan, and deployment gate.

```
45 | sudo sh -s -- -b /usr/local/bin
46 trivy fs \
47 --scanners vuln,misconfig \
48 --severity HIGH,CRITICAL \
49 --ignore-unfixed \
50 --exit-code 1 \
51 .
52 displayName: SCA and IaC Scan - Trivy
53
54 - script: |
55   docker run --rm \
56     -v "$(Build.SourcesDirectory)/repo" \
57     trufflesecurity/trufflehog:latest \
58     filesystem /repo \
59     --results=verified,unknown \
60     --fail
61 displayName: SecretScan - TruffleHog
62
63 - script: |
64   docker build -t $(imageName)$(Build.BuildId) .
65 displayName: Build Docker Image
66
67 - script: |
68   trivy image \
69     --severity HIGH,CRITICAL \
70     --ignore-unfixed \
71     --exit-code 1 \
72     $(imageName)$(Build.BuildId)
73 displayName: Container Scan - Trivy
74
75 - stage: Deploy
76 displayName: Deployment Gate
77 dependsOn: BuildAndSecurity
78 condition: and(succeeded(), or(eq(variables['Build.SourceBranch'], 'refs/heads/main'), eq(variables['Build.SourceBranch'], 'refs/heads/master'))))
79 jobs:
80 - job: DeploymentPlaceholder
81 displayName: Deployment Placeholder
82 steps:
83 - script: |
84   echo "All security checks passed."
85   echo "Next, connect this stage to Azure Container Registry and Azure App Service or Azure Container Apps."
86 displayName: Ready for Azure Deployment
87
```